

Lab 05 Report: CPU Scheduling Simulator

Oct 5, 2026

Skyler Fokine

Student information

Field	Entry
Name	Skyler Fokine
Course	CS 4420 Operating Systems
Assignment	Lab 05: Implementing a CPU Scheduling Simulator

Overview

This program simulates three CPU scheduling algorithms (FCFS, Round Robin, and non-preemptive SJF) one time unit at a time. It reads tasks from an input file, prints each change in task state, and reports per-task statistics and the average waiting time.

Algorithm	How it picks the next task	Preemptive?
FCFS	Front of the ready queue	No
SJF	Ready task with the smallest burst time	No
RR	Front of the ready queue, for at most one time quantum	Yes

Waiting time for each task is computed as end time minus arrival time minus burst time.

How to compile and run

The program is a single C file, `main.c`, built with `gcc` on Linux.

1. Compile:

```
gcc -Wall -Wextra -o main main.c
```

2. Run with an input file and an algorithm. RR also requires a time quantum as the 4th argument.

```
./main input/input.1 FCFS
./main input/input.1 SJF
./main input/input.1 RR 5
```

The input file's first line is the number of tasks, followed by one pid arrival_time burst_time line per task.

Design and tie-breaking rules

The simulator is time-driven: each loop iteration is one time unit. Each unit it adds arrivals to the ready queue, dispatches a task if the CPU is free, runs that task for one unit, and prints any state change. The ready queue is an array of task indexes, capped at 20 per the handout.

The handout does not specify these cases, so the program uses the following rules:

Situation	Rule used
Two tasks arrive at the same time	Queued in input file order
SJF: two ready tasks have the same burst	Earlier arrival first, then lower pid
RR: a quantum expires at the same moment a new task arrives	New arrival is queued first; the preempted task goes behind it
RR: a quantum expires and the queue is empty	The same task runs again and is listed as a new row

Test cases

Four test cases cover the handout sample and three edge cases. Run each file with all three algorithms and paste one screenshot per run.

Test case 1: Handout sample (input.1)

Verifies basic correctness against the handout's published results. P0 is a long job that arrives first, which shows the FCFS convoy effect.

```
4
0 0 12
1 2 4
2 3 1
3 4 2
```

Algorithm	Command
FCFS	./main input/input.1 FCFS
SJF	./main input/input.1 SJF
RR	./main input/input.1 RR 5

Screenshot: FCFS run

```

miko@pop-os:~/classes/OperatingSystemsCouse/hw5$ ./main input/input.1 FCFS
Time 0: P0 arrives
Time 0: P0 starts running
Time 2: P1 arrives
Time 3: P2 arrives
Time 4: P3 arrives
Time 12: P0 finishes
Time 12: P1 starts running
Time 16: P1 finishes
Time 16: P2 starts running
Time 17: P2 finishes
Time 17: P3 starts running
Time 19: P3 finishes

FCFS:
PID   Arrival Time   Start Time   End Time   Running Time   Waiting Time
0      0                0            12         12             0
1      2                12           16         4              10
2      3                16           17         1              13
3      4                17           19         2              13

Average Waiting Time: 9.00

```

Screenshot: SJF run

```
miko@pop-os:~/classes/OperatingSystemsCouse/hw5$ ./main input/input.1 SJF
```

```
Time 0: P0 arrives  
Time 0: P0 starts running  
Time 2: P1 arrives  
Time 3: P2 arrives  
Time 4: P3 arrives  
Time 12: P0 finishes  
Time 12: P2 starts running  
Time 13: P2 finishes  
Time 13: P3 starts running  
Time 15: P3 finishes  
Time 15: P1 starts running  
Time 19: P1 finishes
```

SJF:

PID	Arrival Time	Start Time	End Time	Running Time	Waiting Time
0	0	0	12	12	0
2	3	12	13	1	9
3	4	13	15	2	9
1	2	15	19	4	13

Average Waiting Time: 7.75

Screenshot: RR run

```
miko@pop-os:~/classes/OperatingSystemsCouse/hw5$ ./main input/input.1 RR 5
Time 0: P0 arrives
Time 0: P0 starts running
Time 2: P1 arrives
Time 3: P2 arrives
Time 4: P3 arrives
Time 5: P0 quantum expires
Time 5: P1 starts running
Time 9: P1 finishes
Time 9: P2 starts running
Time 10: P2 finishes
Time 10: P3 starts running
Time 12: P3 finishes
Time 12: P0 starts running
Time 17: P0 quantum expires
Time 17: P0 starts running
Time 19: P0 finishes

RR (Time quantum = 5):
PID   Start Time  End Time  Running Time
0      0           5          5
1      5           9          4
2      9          10          1
3     10          12          2
0     12          17          5
0     17          19          2

PID   Arrival Time  Running Time  End Time  Waiting Time
0      0           12          19         7
1      2            4           9         3
2      3            1          10         6
3      4            2          12         6

Average Waiting Time: 5.50
```

Test case 2: CPU goes idle (input.2)

Verifies the simulator prints an idle message when the ready queue is empty mid-run. P0 finishes at t=3 but P1 does not arrive until t=6.

```
3
0 0 3
1 6 2
2 7 4
```

Algorithm	Command
FCFS	./main input/input.2 FCFS
SJF	./main input/input.2 SJF
RR	./main input/input.2 RR 2

Screenshot: FCFS run

```
miko@pop-os:~/classes/OperatingSystemsCouse/hw5$ ./main input/input.2 FCFS
Time 0: P0 arrives
Time 0: P0 starts running
Time 3: P0 finishes
Time 3: idle
Time 6: P1 arrives
Time 6: P1 starts running
Time 7: P2 arrives
Time 8: P1 finishes
Time 8: P2 starts running
Time 12: P2 finishes

FCFS:
PID  Arrival Time  Start Time  End Time  Running Time  Waiting Time
0      0           0           3         3             0
1      6           6           8         2             0
2      7           8          12         4             1

Average Waiting Time: 0.33
```

Screenshot: SJF run

```
miko@pop-os:~/classes/OperatingSystemsCouse/hw5$ ./main input/input.2 SJF
Time 0: P0 arrives
Time 0: P0 starts running
Time 3: P0 finishes
Time 3: idle
Time 6: P1 arrives
Time 6: P1 starts running
Time 7: P2 arrives
Time 8: P1 finishes
Time 8: P2 starts running
Time 12: P2 finishes

SJF:
PID  Arrival Time  Start Time  End Time  Running Time  Waiting Time
0      0           0           3         3             0
1      6           6           8         2             0
2      7           8          12         4             1

Average Waiting Time: 0.33
```

Screenshot: RR run

```
miko@pop-os:~/classes/OperatingSystemsCouse/hw5$ ./main input/input.2 RR 2
Time 0: P0 arrives
Time 0: P0 starts running
Time 2: P0 quantum expires
Time 2: P0 starts running
Time 3: P0 finishes
Time 3: idle
Time 6: P1 arrives
Time 6: P1 starts running
Time 7: P2 arrives
Time 8: P1 finishes
Time 8: P2 starts running
Time 10: P2 quantum expires
Time 10: P2 starts running
Time 12: P2 finishes

RR (Time quantum = 2):
PID   Start Time  End Time  Running Time
0      0           2           2
0      2           3           1
1      6           8           2
2      8          10           2
2     10          12           2

PID   Arrival Time  Running Time  End Time  Waiting Time
0      0              3             3           0
1      6              2             8           0
2      7              4            12           1

Average Waiting Time: 0.33
```

Test case 3: Simultaneous arrivals (input.3)

Verifies queuing order when every task arrives at $t=0$, and SJF tie-breaking between P1 and P2 (both burst 3).

```
4
0 0 5
1 0 3
2 0 3
3 0 1
```

Algorithm	Command
FCFS	<code>./main input/input.3 FCFS</code>
SJF	<code>./main input/input.3 SJF</code>
RR	<code>./main input/input.3 RR 2</code>

Screenshot: FCFS run

```
miko@pop-os:~/classes/OperatingSystemsCouse/hw5$ ./main input/input.3 FCFS
Time 0: P0 arrives
Time 0: P1 arrives
Time 0: P2 arrives
Time 0: P3 arrives
Time 0: P0 starts running
Time 5: P0 finishes
Time 5: P1 starts running
Time 8: P1 finishes
Time 8: P2 starts running
Time 11: P2 finishes
Time 11: P3 starts running
Time 12: P3 finishes

FCFS:
PID  Arrival Time  Start Time  End Time  Running Time  Waiting Time
0    0              0           5         5             0
1    0              5           8         3             5
2    0              8          11         3             8
3    0             11          12         1            11

Average Waiting Time: 6.00
```

Screenshot: SJF run

```
miko@pop-os:~/classes/OperatingSystemsCouse/hw5$ ./main input/input.3 SJF
Time 0: P0 arrives
Time 0: P1 arrives
Time 0: P2 arrives
Time 0: P3 arrives
Time 0: P3 starts running
Time 1: P3 finishes
Time 1: P1 starts running
Time 4: P1 finishes
Time 4: P2 starts running
Time 7: P2 finishes
Time 7: P0 starts running
Time 12: P0 finishes

SJF:
PID  Arrival Time  Start Time  End Time  Running Time  Waiting Time
3    0              0           1         1             0
1    0              1           4         3             1
2    0              4           7         3             4
0    0              7          12         5             7

Average Waiting Time: 3.00
```


Screenshot: RR run

```
miko@pop-os:~/classes/OperatingSystemsCouse/hw5$ ./main input/input.3 RR 2
Time 0: P0 arrives
Time 0: P1 arrives
Time 0: P2 arrives
Time 0: P3 arrives
Time 0: P0 starts running
Time 2: P0 quantum expires
Time 2: P1 starts running
Time 4: P1 quantum expires
Time 4: P2 starts running
Time 6: P2 quantum expires
Time 6: P3 starts running
Time 7: P3 finishes
Time 7: P0 starts running
Time 9: P0 quantum expires
Time 9: P1 starts running
Time 10: P1 finishes
Time 10: P2 starts running
Time 11: P2 finishes
Time 11: P0 starts running
Time 12: P0 finishes

RR (Time quantum = 2):
PID   Start Time  End Time  Running Time
0      0           2         2
1      2           4         2
2      4           6         2
3      6           7         1
0      7           9         2
1      9          10         1
2     10          11         1
0     11          12         1

PID   Arrival Time  Running Time  End Time  Waiting Time
0      0             5             12         7
1      0             3             10         7
2      0             3             11         8
3      0             1              7         6

Average Waiting Time: 7.00
```

Test case 4: Arrival at quantum expiry (input.4)

Verifies the RR tie rule: at $t=2$, P0's quantum expires at the same moment P1 arrives, so P1 is queued ahead of P0.

3

0 0 4

1 2 3

2 4 2

Algorithm	Command
FCFS	<code>./main input/input.4 FCFS</code>
SJF	<code>./main input/input.4 SJF</code>
RR	<code>./main input/input.4 RR 2</code>

Screenshot: FCFS run

```
miko@pop-os:~/classes/OperatingSystemsCouse/hw5$ ./main input/input.4 FCFS
Time 0: P0 arrives
Time 0: P0 starts running
Time 2: P1 arrives
Time 4: P0 finishes
Time 4: P2 arrives
Time 4: P1 starts running
Time 7: P1 finishes
Time 7: P2 starts running
Time 9: P2 finishes

FCFS:
PID  Arrival Time  Start Time  End Time  Running Time  Waiting Time
0     0             0           4         4             0
1     2             4           7         3             2
2     4             7           9         2             3

Average Waiting Time: 1.67
```

Screenshot: SJF run

```
miko@pop-os:~/classes/OperatingSystemsCouse/hw5$ ./main input/input.4 SJF
Time 0: P0 arrives
Time 0: P0 starts running
Time 2: P1 arrives
Time 4: P0 finishes
Time 4: P2 arrives
Time 4: P2 starts running
Time 6: P2 finishes
Time 6: P1 starts running
Time 9: P1 finishes

SJF:
PID  Arrival Time  Start Time  End Time  Running Time  Waiting Time
0     0             0           4         4             0
2     4             4           6         2             0
1     2             6           9         3             4

Average Waiting Time: 1.33
```

Screenshot: RR run

```
miko@pop-os:~/classes/OperatingSystemsCouse/hw5$ ./main input/input.4 RR 2
```

```
Time 0: P0 arrives  
Time 0: P0 starts running  
Time 2: P0 quantum expires  
Time 2: P1 arrives  
Time 2: P1 starts running  
Time 4: P1 quantum expires  
Time 4: P2 arrives  
Time 4: P0 starts running  
Time 6: P0 finishes  
Time 6: P2 starts running  
Time 8: P2 finishes  
Time 8: P1 starts running  
Time 9: P1 finishes
```

```
RR (Time quantum = 2):
```

PID	Start Time	End Time	Running Time
0	0	2	2
1	2	4	2
0	4	6	2
2	6	8	2
1	8	9	1

PID	Arrival Time	Running Time	End Time	Waiting Time
0	0	4	6	2
1	2	3	9	4
2	4	2	8	2

```
Average Waiting Time: 2.67
```